

TechForce Take-Home: Candidate Import & Scoped Recruiter View

Role: Full-Stack Engineer (Python + React) — Bold Business **Time limit:** 3 hours of recorded working time (see Recording Protocol below) **Deadline:** Submit within 5 calendar days of receiving this brief

Why this test looks the way it does

We are an AI-forward team. You are **expected and encouraged to use AI coding tools** (Claude Code, Cursor, Copilot, Aider, or your own setup) throughout this exercise — that is part of what we're evaluating. We are grading how you direct and review those tools, your architectural judgment, and what you catch that the AI gets wrong. Not your typing speed.

Because of that, this test is completed on your own machine, on your own time, **while recording your screen** so we can see your actual workflow.

Recording Protocol (required — unrecorded submissions are not reviewed)

1. Create a meeting on Zoom, Google Meet, or Microsoft Teams and **join it alone** — no other participants.
2. Turn on **screen sharing for your entire screen** (not a single window), so your editor, terminal, browser, and AI tools are all visible.
3. **Start recording** (local or cloud, whichever your account supports) before you write your first line of code, and keep recording until you are done.
4. At the start of the recording, **state your full name, the date, and "Bold Business take-home"** out loud.
5. Work as you normally would. Narration is optional but welcome — thinking out loud helps us follow your decisions.
6. If you need a break, pause the recording and say so when you resume ("resuming after a break"). Total recorded working time must stay within 3 hours. Multiple recording files are fine if your tool splits them — label them Part 1, Part 2, etc.

7. Do a **1–2 minute test recording first** and play it back to confirm your screen and audio actually captured. Technical failures ("the recording didn't save") mean redoing the test.
8. Submit the recording as a cloud link (Zoom cloud link, Google Drive, Loom upload of the file — anything we can stream) along with your repo.

What we watch for in the recording: how you scope work before prompting, how precisely you instruct your AI tools, whether you read diffs before running them, how you debug when something breaks, and whether your tests actually run and pass on camera.

The Scenario

TechForce is a recruiting operations platform. You're building a small vertical slice of it: importing a messy candidate CSV into PostgreSQL, then exposing that data through an API and UI **where recruiters can only see candidates in their assigned regions.**

You are given two files:

- `candidates_import.csv` — a raw export from a legacy system. It is intentionally messy: duplicates, invalid values, inconsistent formats, and at least one actively hostile row. Treat it the way you'd treat real client data.
- `recruiters.csv` — the recruiter roster. `assigned_regions` is pipe-delimited; `*` means the user is an admin with global visibility.

Canonical region codes are: `US-MW`, `US-NE`, `US-SE`, `US-W`, `PH-MNL`, `CO-BOG`. Anything that can't be confidently normalized to one of these is invalid.

Part 1 — Ingestion (Python / FastAPI / PostgreSQL)

Build `POST /api/v1/candidates/import` that accepts the CSV and loads it into PostgreSQL (local install or Docker — include a `docker-compose.yml` or setup instructions).

Requirements:

1. **Validate with Pydantic v2.** Define what a valid candidate is. Invalid rows are rejected, not silently dropped — the endpoint returns an import report: `{imported: n, merged: n, rejected: [{row, reason}]}`.
2. **Normalize:** emails (case, whitespace), region codes, statuses (define your canonical enum), dates (define which formats you accept and what you do with impossible ones), salary (numeric or null — document your rule).

3. **Deduplicate.** There are exact duplicates, same-email-different-details records, and one candidate_id collision. Choose and **document your merge/conflict rules** — we care more that the rules are explicit and consistently applied than which rule you picked.
4. **SQL injection must be impossible.** Parameterized queries or ORM only. Some rows will test this.
5. **Batch your writes.** No row-at-a-time inserts inside a loop.

Part 2 — Scoped Read API (authorization in the data layer)

Build `GET /api/v1/candidates?search=&status=` with pagination.

- The caller identifies as a recruiter via an `X-Recruiter-Id` header (simplified auth — no login flow needed).
- A recruiter must receive **only candidates in their assigned regions**. Admins (*) see everything.
- **The scoping must be enforced in the query/data layer.** Returning everything and filtering in the client, or filtering after fetch in Python-side application code that could be bypassed by another endpoint, does not count. If you know how to demonstrate this with PostgreSQL Row-Level Security, that's a plus — but a correctly scoped query layer is acceptable.
- Unknown recruiter IDs get a 403, not an empty 200.

Part 3 — Recruiter View (React / TypeScript)

A single-page table view:

- Candidate table (name, email, region, role, status, applied date) backed by your Part 2 API.
- Text search and a status filter.
- A recruiter switcher (dropdown of the 5 recruiters) that changes the `X-Recruiter-Id` sent — so we can flip between Caleb and Marisol and watch the data change.
- Unicode names must render correctly.

Keep styling minimal. We're not grading CSS.

Part 4 — Tests

`pytest` (with `pytest-asyncio` if you go async) covering at minimum:

- A dedup case, an invalid-email rejection, and the injection rows.

- Authorization: assert that a specific recruiter **cannot** retrieve a candidate outside their regions, and that an admin can.
- Run the suite on camera before you stop recording.

Part 5 — NOTES.md

A short write-up (half a page to a page):

- Your dedup/merge rules and normalization decisions.
 - Your **AI usage log**: what you delegated to your tools, what you deliberately did not, and at least one concrete thing the AI produced that you caught and corrected during this exercise.
 - What you'd do next with more time.
-

Submission Checklist

- ☐ Git repo (GitHub link or zip) with meaningful commit history — not one giant commit
- ☐ **README.md** with setup steps that work from a clean machine
- ☐ **NOTES.md** as described above
- ☐ Recording link(s), streamable
- ☐ Test suite passing (shown in the recording)

Ground Rules

- AI tools: unlimited and encouraged. Other humans: not allowed.
- You may use any libraries you like within Python/FastAPI/React/TypeScript/PostgreSQL.
- If something in this brief is ambiguous, make a reasonable call and document it in NOTES.md — that's the job.
- Questions before you start recording are fine; email us.